Understanding the Impact of Proposer Location in Geo-Distributed BFT Consensus

Lucca Cézar[1], Fernando Dotti[1], and Fernando Pedone[2]

[1] PUCRS - School of Technology - PPGCC, Porto Alegre, Brazil
lucca.dornelles99@edu.pucrs.br, fernando.dotti@pucrs.br
[2] Università della Svitzzera italiana, Lugano, Switzerland
fernando.pedone@usi.ch

Abstract. Geo-distributed Byzantine fault-tolerant (BFT) consensus protocols rely on rotating proposers to ensure fairness and distribute responsibilities among validators. This paper presents an experimental study of proposer location in geo-distributed Tendermint deployments. Depending solely on the selected proposer, consensus latency can vary by up to 40%. Motivated by these findings, we introduce a generic, Byzantine-resilient methodology for performance-aware proposer prioritization. The proposed mechanism continuously adapts the proposer frequency based on observed consensus performance without introducing additional communication rounds, reducing the Byzantine fault threshold, or requiring modifications to the underlying consensus protocol. We also present a taxonomy of existing performance-aware leader-selection strategies and position our approach within this design space. Although instantiated for Tendermint, the proposed methodology is applicable to a broader class of rotating-leader state machine replication protocols.

Keywords: Byzantine consensus · Blockchains · Leader selection.

1 Introduction

Modern Byzantine fault-tolerant (BFT) consensus protocols increasingly target geo-distributed deployments, where validators are spread across multiple regions and datacenters. In such environments, wide-area network latency becomes a dominant component of consensus execution time, making network topology a critical determinant of performance.

In BFT protocols that employ *rotating proposers* (e.g., [5]), wide-area network latency plays a particular role. In such protocols, in each consensus instance, the proposer, a designated validator, is responsible for injecting the next value (e.g., a block of transactions) into the consensus protocol by disseminating it to the remaining validators. Rotating proposers improves fairness, reduces opportunities for censorship, and distributes rewards among validators [1]. Therefore, it is unsurprising that proposer selection has received significant attention as a means of improving performance. Comparatively little is understood, however, about how the proposer's position within the network topology affects the execution of consensus itself.

Understanding this relationship is important not only for characterizing the behavior of modern BFT protocols, but also for determining whether topology-aware proposer selection can improve performance without sacrificing Byzantine resilience. Our experimental study of geo-distributed Tendermint deployments reveals that proposer location is a first-order determinant of consensus performance. Depending solely on the selected proposer, overall consensus latency can vary by up to 40%. Moreover, the impact of proposer location does not end once the proposal has been disseminated. Because nodes closer to the proposer receive the proposal earlier, they are more likely to form the first quorums, which, in turn, influence the execution of subsequent consensus phases. These observations naturally suggest that proposer selection should account for network topology.

Motivated by this insight, we propose a generic, performance-aware proposer-prioritization mechanism for rotating-leader consensus protocols. Rather than removing validators from the committee or modifying the underlying consensus algorithm, our mechanism continuously adjusts proposer frequency based on the observed durations of previous consensus instances. The mechanism is decentralized, introduces no additional communication rounds, preserves the Byzantine fault threshold, and can be integrated with existing rotating-leader protocols with minimal modifications.

The main contributions of this paper are as follows:

- We present a taxonomy of performance-aware leader selection strategies for Byzantine fault-tolerant consensus. We classify existing approaches by their design choices, highlight common trade-offs among performance, resilience, protocol integration, and fairness, and position our work within this design space.
- We provide the first experimental characterization of the impact of proposer location on performance in geo-distributed Byzantine consensus. Our evaluation shows that the choice of the proposer significantly influences consensus latency, quorum formation, and the duration of individual protocol phases, revealing performance differences of up to 40% that arise solely from network topology.
- We propose a generic, Byzantine-resilient methodology for performance-aware proposer prioritization. The proposed mechanism continuously adapts proposer frequency based on observed consensus performance, without introducing additional communication rounds or reducing the Byzantine fault threshold. We also argue for the correctness of the proposed approach.

The remainder of this paper is organized as follows. Section 2 introduces the system model and provides the necessary background on Tendermint. Section 3 reviews related work and proposes a taxonomy of leader selection strategies. Section 4 experimentally characterizes the impact of proposer location on consensus execution. Section 5 presents the proposed proposer prioritization mechanism and discusses its correctness. Section 6 concludes the paper and outlines directions for future work.

2 Background

2.1 System Model

We consider a distributed system composed of an unbounded set of client processes that submit transactions (or operations) to a bounded set Π of N server processes. For simplicity, here we consider that all nodes in Π act as *validators* (i.e., nodes that execute the consensus protocol). The set of nodes is dynamic and known by all nodes.

Nodes communicate by message passing. The system is partially synchronous: it is initially asynchronous when there is no bound on message delays or relative process speeds. At an unknown moment (i.e., the Global Stabilization Time), the system becomes synchronous, when message delays and processing speed become bounded.

There are up to $F < N/3$ *faulty* or Byzantine nodes; the remaining nodes are *honest*. Faulty nodes can deviate from their specifications, whereas honest nodes adhere to them. Cryptographic techniques are used, with adversaries not having (with high probability) the computational power to subvert them.

2.2 Blockchain Consensus

We consider permissioned consensus protocols, such as Practical Byzantine Fault Tolerance (PBFT) [2] and Tendermint [1]. Tendermint consensus operates through three sequential phases, *propose*, *prevote*, and *precommit*, which functionally correspond to the *pre-prepare*, *prepare*, and *commit* stages of the PBFT algorithm. The three phases work as follows:

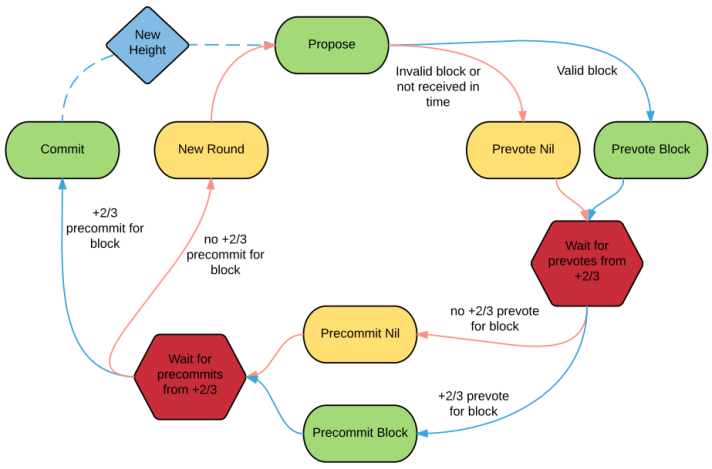

Fig. 1: Basic Tendermint operation [1]

- **Propose**
  The deterministic leader selection function indicates to each node which member of the committee is the current proposer. If a node determines that it is the leader of the current consensus instance it must generate a block, based on its received transactions, and propose it to the network.
- **Prevote**
  Once a node receives a proposal it analyzes the block's validity. If the block is valid and was sent from the correct proposer, the node agrees with the proposal and broadcasts a prevote for the block. If the block is invalid, the node prevotes nil.
  As proposers may not send proposals in a timely manner, due to network issues or Byzantine faults, Tendermint makes use of timeouts. After processing the block, each node awaits a dynamic time interval. If the proposal is not received within this interval, the node automatically prevotes nil. This timeout is calculated locally according to pre-defined rules, allowing liveness to be maintained as long as the model assumptions hold.
  After sending its prevote, each node awaits for a quorum. If the node receives a quorum of prevotes for the block, it precommits the block. In any other case, including if the node times out when waiting for a quorum, the node precommits nil.
- **Precommit**
  The precommit phases begins as soon as the node broadcasts its precommit value. If the node receives a quorum of precommits for the block, the block is considered committed and appended to the chain. Otherwise, if a quorum cannot be reached or if the node timeouts, consensus restarts with the next proposer.

A primary distinction of Tendermint is the implementation of a rotating proposer mechanism: by using a round-robin approach in which each node proposes only once, the network mitigates censorship [9].

Furthermore, unlike PBFT, Tendermint is designed for environments where unknown participants can join. To protect against Sybil attacks [3], where a malicious actor might otherwise flood the network with nodes to gain control, the protocol integrates a Proof of Stake (PoS) mechanism. Participants receive rewards for consensus participation, while detected malicious behavior results in the destruction or reduction of their stake via slashing. This design effectively increases the cost of subversion, making it expensive for an adversary to acquire a quorum.

3 Taxonomy of leader selection strategies

Several works have explored performance-aware leader selection in Byzantine fault-tolerant consensus. Existing proposals differ substantially in both their objectives and integration strategies. To better position our contribution, we classify prior work along seven dimensions (see Table 1): **(I) Score Representation**, distinguishing between continuous scores and discrete node classes;

(II) Committee Eviction, indicating whether nodes may be removed from the active committee; **(III) Evaluation Metrics**, describing the information used to estimate node quality; **(IV) Communication Overhead**, indicating whether the mechanism requires additional protocol messages or coordination phases; **(V) Decoupled Architecture**, capturing whether the optimization is implemented independently from the consensus core and can therefore be reused across different SMR protocols; **(VI) Continuous Evaluation**, indicating whether node quality is continuously updated during execution; and **(VII) Rotation Policy**, describing whether proposer rotation remains active after optimization.

Most related proposals are derived from PBFT-style primary-backup architectures and therefore focus on improving view-change behavior after detecting a slow or faulty leader (e.g., [5, 8, 12, 14]). In these systems, node differentiation primarily serves as a ranking mechanism for selecting replacement leaders after a disruption. Proposer optimization is often reactive rather than adaptive.

A second recurring characteristic is the reliance on committee eviction to sustain performance (e.g., [4, 6, 7, 10, 13]). These approaches remove nodes considered slow, unstable, or suspicious from the active consensus set. While effective at increasing throughput in controlled environments, committee reduction introduces a direct trade-off between performance and Byzantine resilience. Removing slow but correct nodes reduces the effective committee size, thereby lowering the maximum number of Byzantine faults the system can tolerate. Moreover, modern blockchain deployments already incorporate mechanisms to detect and exclude malicious validators, making additional performance-oriented eviction layers potentially redundant and operationally risky.

The surveyed literature also differs in how it treats proposer fairness. Many PBFT-derived optimizations effectively converge toward fixed or weakly rotating leaders, increasing susceptibility to censorship and centralization. Our approach instead adopts proportional prioritization: higher-performing nodes are selected more frequently, while all nodes remain eligible proposers. This preserves continuous rotation while still exploiting topology-aware performance asymmetries.

Finally, existing work generally treats proposer performance as an abstract property associated with computational speed or node responsiveness, with limited investigation into how network topology shapes consensus execution. Our experiments show that proposer placement directly affects overall consensus latency, influencing the duration of subsequent protocol phases. In geo-distributed deployments, proposer location therefore becomes a first-order determinant of consensus efficiency rather than merely a secondary implementation detail.

Table 1 highlights two dominant trends in prior work: (i) performance optimization through committee reduction, and (ii) tightly coupled protocol-specific leader management. Our approach differs in that it preserves the full committee and the original Byzantine threshold while continuously adapting proposer frequency based on observed consensus performance. Unlike eviction-based strategies, proposer prioritization improves throughput without sacrificing resilience or requiring invasive protocol modifications.

Algorithm	Score Type	Committee Eviction	Evaluation Metrics	Communic. Overhead	Decoupled Architecture	Continuous Evaluation	Rotation Policy
[4]	C	Y	Consensus participation and proposal success ratio	Y	Y	Y	N
[5]	C	N	Per-node consensus latency measured through active probing by clients	Y[3]	Y	N	N
[6]	D	Y	Composite reputation derived from performance and participation	Y	N	Y	N
[7]	D	Y	Latency, availability, activity level, and consensus contribution	Y	N	Y	N
[8]	D	N	Behavioral classification and response-time analysis	Y	N	Y	N
[10]	C	Y	Latency measurements combined with reputation signals	Y	N	Y	N[4]
[11]	C	N	Observed leader throughput during evaluation windows	N	Y	N	N
[12]	C	N	Historical leader success, measured by committed blocks	Y	Y[5]	Y	N
[13]	D	Y	Binary success/failure outcomes of consensus participation	N	N	Y	N
[14]	C	N	Consensus participation consistency, and failure frequency	Y	N	Y	N
This paper	C	N	Consensus duration	Y	Y	Y	P

Table 1: Taxonomy of leader selection strategies.

Legend: Continuous (C) scores allow fine-grained proposer differentiation, while Discrete (D) scores classify nodes into categories. Committee Eviction indicates whether nodes may be removed from the active consensus committee. Proportional (P) rotation biases proposer frequency according to performance, whereas No rotation (N) maintains the primary/proposer until a view change is proposed.

4 Understanding the Impact of Proposer Location

This section investigates how the geographic distribution of consensus nodes affects overall performance. Following an experimental approach, we configure topologies with chosen node-to-node communication latencies, submit a workload, and measure consensus latency at each participating node, under every different proposer node.

[3] Multiple consensus.
[4] The primary is the node with the most voting power [10]. If voting power changes, a new node can become the primary even without suspected failure. However, the algorithm does not perform intentional leader rotation.
[5] For systems without leader rotation.

Environment. The experiments were conducted on the CloudLab[6] testbed using 13 m510[7] nodes. The desired latencies are injected via the `tc`[8] (traffic control) utility. The study utilized the Malachite implementation of the Tendermint protocol. Each execution lasted five minutes, including a 30-second warm-up and 15-second cool-down.

Workload. Instead of creating a set of clients submitting operations to network nodes, we configured Tendermint nodes to submit a standard proposal value whenever they become proposers for the respective height, according to the protocol. This represents the minimum latency case, where proposers instantaneously submit their proposal to the network whenever they are entitled to.

Metrics. Our primary metric for evaluating a proposer's performance is the latency to decide each height proposed by that proposer, measured at each node. As we want these measures to be useful in a monitoring protocol (see Section 5), we use intra-node measurements, which avoids the possibility of dishonest nodes forging fast latency samples in their favor. Moreover, no clock-synchronization procedure is needed. A simplification of the algorithm at each node running Tendermint, annotated with sampling procedures, is shown in Algorithm 1. When a node finishes height $h-1$, it locally enters the proposal phase of height h. This is the moment when the latency sampling is initiated. A proposer should then send the proposal, while a non-proposer waits for the proposal. Afterward, when the decision is reached, the sample is completed. We call this metric the *local consensus duration*.

Topology with uniform latencies among node pairs. We first conducted a sanity check using a 4-node topology with a uniform 100ms latency between any pair of nodes. We measured the local consensus duration for each possible pair of proposer and node. The results showed negligible variation ($< 1.6\%$) across samples, indicating that any subsequent performance discrepancies were strictly due to network topology. Also, we observed no impact in switching proposers at each decision or after a number of decisions. For better legibility, we show figures with results switching proposers every 10 decisions.

Topology configured for geo-distributed latencies. Next, we choose a topology in which each node emulates a distinct AWS zone, with 13 zones, and inter-node latencies.[9] Figure 2 depicts the measurements obtained.

[6] `https://cloudlab.us/`
[7] `https://docs.cloudlab.us/hardware.html#(part.cloudlab-utah)`
[8] https://www.man7.org/linux/man-pages/man8/tc.8.html
[9] `https://github.com/cometbft/cometbft/blob/v2.x/test/e2e/pkg/files/aws-latencies.csv`

Algorithm 1 Simplification of Tendermint at node n_i

```
1: upon Init
2:     h ← 0
3:     trigger proposal phase for instance h
4:
5: upon proposal phase for instance h
6:     startTime_h ← current_time                ▷ Every node locally starts sample h
7:     if n_i is proposer at h then
8:         send proposal
9:
10: upon receive proposal for instance h
11:     ...
12:
13:     protocol unfolds                          ▷ Execution of the consensus protocol
14:     ...
15: upon commit of instance h
16:     sample_h ← current_time − startTime_h     ▷ End of sample h
17:     record sample_h at node's n_i log
18:     commit actions for instance h
19:     h' ← h + 1
20:     trigger proposal phase for instance h'
21:
```

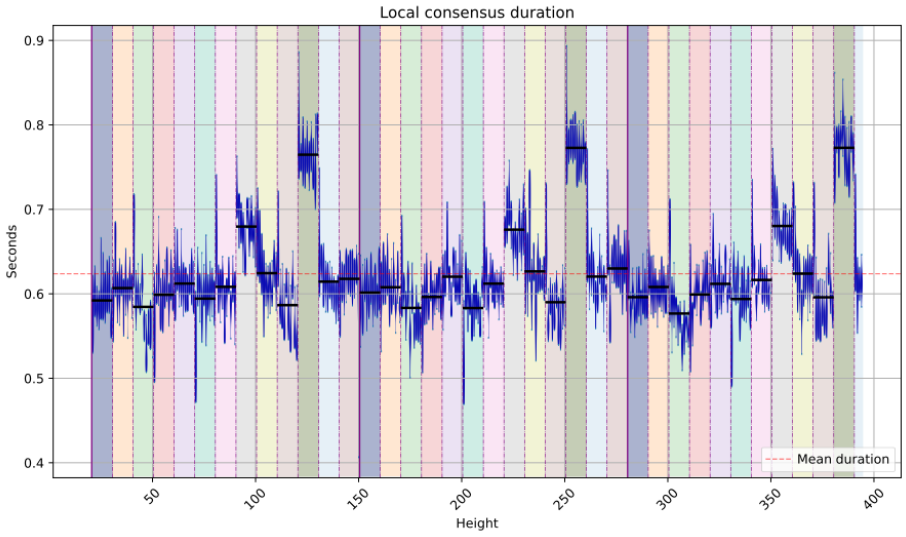

Fig. 2: Local consensus duration under different proposers. Each proposer has a different color, divided by vertical dotted lines. Each proposer proposes 10 instances in a row, then switches to the next. Each vertical blue line represents a consensus instance and is composed of plots of the respective local consensus duration samples collected at each node. Solid lines indicate a new round through the same sequence of proposers, repeating the pattern.

Figure 2 shows, from left to right, 3 rounds over the same 13 proposers, each one with a different color, proposing 10 consensus instances in a row. A first observation is that the collected latencies vary by proposer. Secondly, latencies recur for the same proposers across different rounds. Thirdly, the consensus latency is shown to be related to the respective proposer's communication latency: for instance, the worst performing node (i.e., 11th in each round), corresponds to the node with (i) the highest average communication latency to all other nodes; (ii) the highest average communication latency considering a quorum (i.e., the $2f + 1 = 9$ closest nodes, with lower latencies); and (iii) the highest latency considering the slowest node in a quorum (i.e., the 9th node ordered by latency).

Proposer impact on quorum formation. After the proposal, further communication phases are all-to-all. While the time to receive a quorum, in all-to-all communication phases, theoretically depends on the location of each receiver node, we investigated whether the proposer's location could influence such steps. To check this, we computed and evaluated the quorums formed at each node, rotating the entire set of proposers over several rounds. We observed 98% consistency in each node's local quorum composition when the same node served as proposer. Nodes geographically closer to the proposer receive the proposal faster and, consequently, their votes are the first to arrive and form the *prevote* quorum.

As stated in the Tendermint specification, if a farther node manages to join another node's prevote quorum, that second node will wait for the node in the *precommit* step, even if a faster quorum is reached (this is intended for Proof of Stake rewards). This means that in the precommit phase, a specific node cannot progress with its ideal quorum (set of closest nodes), negatively impacting latency and throughput. This also leads to the high consistency among prevote and precommit quorums for a specific node.

These results demonstrate that the proposer has a considerable effect on both consensus duration and step durations across all nodes in the network. It is expected that, in linearized protocols such as Hotstuff, this impact could be even more pronounced due to the centrality of the proposer (leader). Also, the above experiments show that intra-node measurements allow us to differentiate the performance of different proposers.

5 Performance-Aware Proposer Prioritization

We propose a generic mechanism that continuously adapts the proposer frequency based on the observed performance of consensus instances. The mechanism targets consensus protocols that employ rotating proposers and maintain a deterministic proposer-selection function, denoted by *nextProposer* (e.g., derived from stake in proof-of-stake systems). The approach operates in three stages: nodes first locally classify proposers according to observed consensus latency (Section 5.1); these observations are then reconciled into a globally consistent

classification history (Section 5.2); finally, the resulting classifications are incorporated into proposer selection, biasing proposer frequency toward nodes that consistently lead to faster consensus decisions while preserving continuous rotation and Byzantine resilience (Section 5.3). We conclude the section by discussing the properties and correctness of the proposed mechanism (Section 5.4).

5.1 Sampling and classifying proposers

Each node continuously monitors the latency of consensus instances in order to estimate the relative performance of proposers. This is carried out using the procedure explained in Section 4, Algorithm 1. For each consensus height, each node locally measures the elapsed time from the start of the consensus instance to the final decision.

Since each consensus instance is associated with a known proposer, the observed latency provides an estimate of that proposer's effectiveness at driving consensus progress.

Each node maintains a sliding window L containing the last k observed consensus latencies, where k is a configuration parameter. From this window, the node continuously computes the average consensus latency $\bar{L}$. A proposer is then classified relative to this average latency.

Given a latency sample s, the proposer is classified as:

$$fast \quad \text{if } s \leq \bar{L}$$

$$slow \quad \text{if } s > \bar{L}$$

We denote by $c_h \in \{fast, slow\}$ the resulting classification for a consensus height h.

5.2 From local to global classifications

We now show how to transform local classifications into a globally consistent classification that can later be used to bias proposer selection. Rather than introducing a separate consensus protocol, our approach piggybacks classification information onto the existing consensus execution. The mechanism operates continuously through three lightweight steps: *vote*, *propose*, and *adopt*.

Vote step. Once the node reaches a decision on height h, the node calculates the consensus duration and compares it to the average to obtain its local c_h vote for that height. This value is only calculated once, and an honest node will never change a previous vote for any reason. The node then sends its local c_h along with the first consensus message (e.g., Tendermint's `prevote`) for the height $h+1$, thereby avoiding additional communication rounds. This vote includes the height it refers to, its vote, and a cryptographic signature.

Choose step. At the beginning of height $h+2$, the proposer evaluates the classifications received for the proposer of height h, reaching one of three states

1. The proposer receives $2F+1$ votes for a decision $d_h \in \{fast, slow\}$, in which case it proposes d_h.
2. The proposer receives at least $F+1$ votes for $fast$ and at least $F+1$ votes for slow, making a decision impossible. In this case, the proposer proposes $\perp$ (non-decision).
3. The proposer does not receive enough votes to reach a decision. Consensus proceeds as normal, but without any decision attached.

In cases 1 and 2, the proposer must send valid signatures to validate its proposal. The lack of or invalid signatures renders the block invalid.

Adopt step. After a consensus with a valid $d_h \in \{fast, slow\}$ is committed, each node updates its local score accordingly, changing its internal values for the proposer selection algorithm. As a consequence, all honest nodes eventually converge to the same proposer classification history.

To understand the need for $2F+1$ matching classifications, let k be the number of votes the proposer receives, where $k = k_{fast} + k_{slow}$. At most F of these votes are from Byzantine processes, and there are votes from $(N-k)$ nodes that the proposer does not know about. Therefore, to guarantee that there are votes from a majority of honest nodes for *fast* (resp., *slow*), it must be that $k_{fast} - F > k_{slow} + k_{\perp} + (N - k_{fast} - k_{slow})$, or $k_{fast} \geq \lceil (N+F)/2 \rceil$, and since $N = 3F+1$, $k_{fast} \geq 2F+1$.

5.3 Continuous proposer scoring and prioritization

The goal of the scoring mechanism is to slightly bias proposer selection toward nodes that consistently lead to faster consensus decisions, while preserving stake proportionality and ensuring that every node remains eligible to propose.

Each node n maintains a continuous score $score_n \in [-1, 1]$ representing the recent observed performance of a proposer. Scores are continuously updated according to the classifications produced by the mechanism described in the previous section. We define $decision_n \in \{-1, 1\}$, where: $slow = -1$ and $fast = 1$. $\perp$ decisions do not require score updates.

Scores evolve according to the following exponential averaging recurrence:

$$score'_n = trunc_{[-1,1]}(\alpha \cdot decision_n + (1-\alpha) \cdot score_n)$$

where parameter α, $0 \leq \alpha \leq 1$, controls how strongly new observations influence the score. Larger α values make the score react faster to performance changes, whereas smaller α values make scores less sensitive to transient fluctuations. Function $trunc_{[-1,1]}()$ ensures that scores are within interval $[-1, 1]$.

The resulting score is then combined with the node's stake to bias proposer selection:

$$weight_n = stake_n \cdot (1 + C \cdot score_n)$$

where parameter C, $0 \leq C < 1$, controls how strongly observed performance influences proposer selection relative to stake. When $C = 0$, proposer selection depends only on the stake. As C increases, higher-performing nodes become proportionally more likely to be selected as proposers. Since $score_n \in [-1, 1]$ and $C < 1$, every node always retains a strictly positive weight and therefore remains eligible to become a proposer.

The deterministic `nextProposer` function of the underlying consensus algorithm is then executed using $weight_n$ instead of the original $stake_n$. As a result, proposer selection continuously adapts to observed consensus performance while preserving the original proposer rotation mechanism.

Finally, since the system is decentralized, parameters α and C must remain consistent across nodes. In systems that support dynamic reconfiguration, these parameters may themselves be updated via consensus and recorded in the replicated state.

5.4 Properties and correctness

We now argue that the proposed mechanism satisfies the following properties.

We define a *good run* as an execution in which every honest proposer receives classification votes from all honest processes.

- P.1 The safety and liveness properties of the underlying consensus algorithm are preserved.
- P.2 Proposers associated with faster (slower) consensus decisions become proposers more (less) frequently.
- P.3 Changes in the relative performance of proposers over time lead to corresponding adaptations in proposer frequency.
- P.4 Every honest node eventually becomes a proposer.
- P.5 If all honest processes classify proposer P as $x \in \{fast, slow\}$, and the proposer responsible for choosing the classification is honest and executes in a good run, then P is eventually adopted as x.
- P.6 A proposer is adopted as $x \in \{fast, slow\}$ only if at least a majority of honest processes classify it as x.

Argument for P.1. The proposed mechanism does not modify the underlying consensus algorithm. Instead, it piggybacks signed classification votes onto existing protocol messages and processes them independently from consensus execution. Moreover, the mechanism does not introduce any additional coordination that could block consensus progress. Therefore, the safety and liveness properties of the underlying consensus protocol are preserved.

Argument for P.2. The score $score_P$ of proposer P increases when the proposer is classified as *fast*, and decreases when it is classified as *slow* (Section 5.3). Since proposer weights are computed as $weight_P = stake_P \cdot (1 + C \cdot score_P)$, higher-performing proposers eventually receive larger weights and therefore become more likely to be selected by the deterministic *nextProposer* function.

Argument for P.3. The mechanism continuously updates proposer scores using recent consensus observations. Parameter α controls how strongly new classifications affect the score, while older observations are progressively discounted. Consequently, changes in proposer performance over time are reflected in the corresponding weights and proposer frequencies.

Argument for P.4. Every proposer P retains a strictly positive weight since $score_P \in [-1, 1]$ and $C < 1$. Therefore, no proposer is permanently excluded from proposer selection. Since the underlying *nextProposer* mechanism rotates among all positive weights, every honest node eventually becomes a proposer.

Argument for P.5. Let Q be the honest proposer responsible for choosing the classification of proposer P. Without loss of generality, assume that all honest processes classify P as *fast*. Since there are $2F+1$ honest processes and the run is good, Q receives $2F+1$ matching signed classifications for *fast*, independently of any Byzantine votes. Therefore, the condition for choosing a non-$\perp$ classification is satisfied, and Q includes the classification *fast* together with the supporting signed votes in the proposed value. Since the proposal is decided by consensus, all honest processes eventually adopt the classification *fast*. The argument for *slow* is symmetric.

Argument for P.6. Assume proposer P is adopted as *fast*. Then, some proposer Q must have proposed a valid classification supported by at least $\lceil (N+F)/2 \rceil$ matching signed classifications for *fast* (Section 5.2).

Let k_h and k_b denote the number of received *fast* classifications originating from honest and Byzantine processes, respectively. Thus,

$$k_h + k_b \geq \left\lceil \frac{N+F}{2} \right\rceil.$$

Since at most F processes are Byzantine, we have $k_b \leq F$. Therefore,

$$k_h \geq \left\lceil \frac{N+F}{2} \right\rceil - F.$$

Using $N = 3F + 1$, we obtain

$$k_h \geq \left\lceil \frac{4F+1}{2} \right\rceil - F = (2F+1) - F = F + 1.$$

Since there are $2F + 1$ honest processes, at least $F + 1$ honest classifications constitute a majority of honest processes. Therefore, a proposer can only be adopted as *fast* (resp., *slow*) if a majority of honest processes classified it as *fast* (resp., *slow*).

6 Discussion and future work

This paper makes two main contributions. First, we experimentally characterize the impact of proposer location on geo-distributed Byzantine consensus. Our results show that the proposer is a first-order determinant of consensus performance: its position within the network topology influences not only the duration of the proposal phase itself, but also the composition of quorums and, consequently, the duration of subsequent consensus phases. Depending on the topology, the choice of proposer can increase consensus latency by up to 40%, demonstrating that proposer placement is an important yet often overlooked aspect of geo-distributed deployments.

Second, we introduce a generic performance-aware proposer prioritization mechanism that continuously adapts proposer frequency according to observed consensus performance. The mechanism preserves the original Byzantine fault-tolerance guarantees, introduces no additional communication rounds, and can be integrated into existing rotating-leader state machine replication protocols with minimal modifications. Although our evaluation focuses on Tendermint, the proposed approach is independent of any specific consensus protocol and is applicable to a broader class of rotating-leader systems.

Several directions remain for future work. First, we intend to implement the complete proposer prioritization mechanism within the Malachite codebase and perform a large-scale evaluation. Our goal is to deploy experiments on a local cluster with emulated wide-area latencies that reproduce realistic Cosmos network conditions and validator topologies. This will allow us to study the behavior of the mechanism at scales approaching production deployments, which may involve committees with up to 180 validators.[10]

Second, a large-scale evaluation will enable us to investigate several open questions that cannot be fully addressed in the current study. In particular, we intend to analyze the mechanism's sensitivity to its configuration parameters, evaluate its robustness under dynamic network conditions, and assess the trade-offs between fairness and performance when only a subset of the slowest proposers is deprioritized. Such an evaluation will provide a better understanding of the practical limits and benefits of performance-aware proposer selection in real-world blockchain systems.

References

1. Buchman, E.: Tendermint: Byzantine fault tolerance in the age of blockchains. Master's thesis, University of Guelph (2016)
2. Castro, M., Liskov, B., et al.: Practical byzantine fault tolerance. In: OsDI. vol. 99, pp. 173–186 (1999)
3. Douceur, J.R.: The sybil attack. In: International workshop on peer-to-peer systems. pp. 251–260. Springer (2002)

[10] `https://docs.cosmos.network/hub/latest/validators/overview`

4. Du, N., Liang, Z., Huang, Y., Guo, Z., Yang, H., Wang, S.: Performance optimisation method of pbft consensus for supply chain integration svm. In: 2020 7th international conference on dependable systems and their applications (DSA). pp. 371–377. IEEE (2020)
5. Eischer, M., Distler, T.: Latency-aware leader selection for geo-replicated byzantine fault-tolerant systems. In: 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W). pp. 140–145. IEEE (2018)
6. Huang, D., Huang, Y., Yang, Y.: Improved pbft consensus algorithm based on node evaluation and dynamic management. In: Proceedings of the 2023 4th International Conference on Big Data Economy and Information Management. pp. 851–855 (2023)
7. Qiao, P.: Enhancing manufacturing-as-a-service supply chain transparency with master-slave multi-chain and sqs-pbft. IEEE Access **12**, 186605–186616 (2024)
8. Shan, B., Gao, T., Song, L., Cui, Y.: Grouped byzantine fault-tolerant consensus mechanism based on node behaviour analysis. In: 2025 4th International Symposium on Computer Applications and Information Technology (ISCAIT). pp. 1890–1895. IEEE (2025)
9. Wahrstätter, A., Ernstberger, J., Yaish, A., Zhou, L., Qin, K., Tsuchiya, T., Steinhorst, S., Svetinovic, D., Christin, N., Barczentewicz, M., et al.: Blockchain censorship. In: Proceedings of the ACM Web Conference 2024. pp. 1632–1643 (2024)
10. Wen, X., Yang, X.: Mapbft: multilevel adaptive pbft algorithm based on discourse and reputation models. The Computer Journal **68**(6), 635–648 (2025)
11. Yu, L., Wu, Y., Lu, J., Li, T.: An adaptive reputation update mechanism for primary nodes in pbft. In: 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). pp. 1948–1953. IEEE (2024)
12. Zhang, G., Pan, F., Tijanic, S., Jacobsen, H.A.: Prestigebft: Revolutionizing view changes in bft consensus algorithms with reputation mechanisms. In: 2024 IEEE 40th International Conference on Data Engineering (ICDE). pp. 1930–1943. IEEE (2024)
13. Zhao, F., Wang, Y.: Pbft consensus algorithm based on reward and punishment mechanism. In: 2022 3rd International Conference on Information Science and Education (ICISE-IE). pp. 168–173. IEEE (2022)
14. Zhu, X., Hu, X., Zhu, W.: Rgpbft: A reputation-based pbft algorithm with node grouping strategy. Arabian Journal for Science and Engineering **50**(15), 11837–11850 (2025)